

CAN304 Lab 3

Cryptographic Hash Functions and Message Authentication

In Lab 2, you have installed the PyCryptodome library and practiced AES and RSA encryption schemes. This lab will focus on cryptographic hash functions and message authentication codes (MACs), which are fundamental building blocks in modern cryptography.

Dependencies

- Python 3
- PyCryptodome library installed: `pip3 install pycryptodome`

1. Cryptographic Hash Functions

A cryptographic hash function takes an input (message) and returns a fixed-size string of bytes. The output is typically called the "hash value" or "digest." Essential properties include:

- Deterministic: Same input always produces same hash
- Fast Computation: Hash can be computed quickly
- Pre-image Resistance: Given hash output h , it's hard to find any input x such that $\text{hash}(x) = h$
- Collision Resistance: Hard to find two different inputs that produce the same hash
- Avalanche Effect: Small change in input produces significantly different output

Common Hash Algorithms:

- MD5: 128-bit hash (considered broken for cryptographic purposes)
- SHA-1: 160-bit hash (deprecated for most security applications)
- SHA-256: 256-bit hash (part of SHA-2 family, widely used)
- SHA-512: 512-bit hash (stronger version of SHA-2)
- SHA3-256/SHA3-512: SHA-3 standard using sponge construction

Experiment 1: Compare basic hash functions

Run the [basicHash.py](#), you should see

```

1. Basic Hash Computation
Message: Cryptography is essential for modern security
MD5          | 131de05d66dc81f32cf2e6899762fbd6... | 128 bits | 0.1799 ms
SHA-1        | 42e40f71f84786631ca1f6965db58dec... | 160 bits | 0.0474 ms
SHA-256      | bcb2d4f9c536206a07085fda4dc5a558... | 256 bits | 0.0349 ms
SHA-512      | 770492e027eb36b8222d0bfacc37fd4b... | 512 bits | 0.0356 ms
SHA3-256     | 4950c0fa9ef2d976c16386ab311ab479... | 256 bits | 0.0214 ms
SHA3-512     | 1ef7c071f6b174cfae42171609b34be4... | 512 bits | 0.0222 ms

2. Avalanche Effect
Message 1: AAAAAAAAAAAAAAAAAAAAA... (100 'A's)
Message 2: BAAAAAAAAAAAAAAAAAAAA... ('B' + 99 'A's)

Crypto..SHA256:
Hash 1: d82c6aa133a0fc25b087f46ad7ed2a30...
Hash 2: 220f03289d15758d223032e3081a50b7...
| Differing bits: 141/256 (55.08%)

Crypto..SHA512:
Hash 1: 95c8a866272a76315b891c509e54ef3e...
Hash 2: 0d0819188b8dc6f0f73fa3b309097ce9...
Differing bits: 271/512 (52.93%)

Crypto..SHA3_256:
Hash 1: 9cdc6b47b891b911d3e9d846b5ddf961...
Hash 2: 6e1dbef613b79362c68f9de9df6f84c1...
Differing bits: 133/256 (51.95%)

```

Compare the performance (runtime) and observe avalanche effect of different hash functions.

2. HMAC (Hash-based Message Authentication Code)

HMAC is a specific type of MAC (Message Authentication Code) that involves a cryptographic hash function and a secret key. It provides both data integrity and authentication simultaneously.

The HMAC algorithm is defined as:

$$\text{HMAC}(K, m) = H((K' \oplus \text{opad}) \parallel H((K' \oplus \text{ipad}) \parallel m))$$

Where:

- H: Cryptographic hash function (e.g., SHA-256)
- K: Secret key
- K': Derived key (padded if necessary)
- m: Message
- opad: Outer padding (0x5c repeated)
- ipad: Inner padding (0x36 repeated)
- ||: Concatenation
- $\oplus$: XOR operation

HMAC vs Simple Hash

HMAC provides authentication; simple hash only provides integrity.

Experiment 2: HMAC implementation

Run the [HMAC.py](#), you should see:

```
1: Basic Message Authentication
Original Message: Transfer $1000 to account 987654321
Secret Key (base64): 46F3zYxPo0920ALq/y26vKR82ShNkVHsBw6/q+UX28Y=

[SENDER] Computed HMAC-SHA256: 55c849aff6daaa49949f364fdb4300101999a1f4d2e45cdb0307158ea489f7c6

[RECEIVER] Verification Process:
Message authentication successful

2: Message Tampering Detection
Original Message: Critical command: SHUTDOWN system at 0300
Original HMAC: 66b750fa2586f38a2b42bd3d5a44cf435a6347f6be0177240b0da9d62a3b3e30

[TAMPERING DETECTED]
Intercepted Message: Critical command: SHUTDOWN system at 1300
Computed HMAC for received message: 7e487fc57f0a8922d67f51e48dfcfe37c4a5a67e5ab79dbda8e92ed0d25991f1
Stored HMAC for original message: 66b750fa2586f38a2b42bd3d5a44cf435a6347f6be0177240b0da9d62a3b3e30

SECURITY MAINTAINED: Tampering detected and rejected
```

Observe the result and explain

- Why verification on the receiver side passed in the Basic Message Authentication experiment?
- Why the Message Tampering Detection experiment output “Tampering detected and rejected”?